



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: GARCÍA SITIO DIEGO

N° de Cuenta: 318120601

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 09/09/2026

CALIFICACIÓN: _____

EJERCICIO 1

Descripción: Dibujar las iniciales de sus nombres cada letra de un color diferente.

Solución: Se recuperó el código de la práctica anterior para hacer las iniciales de mi nombre, esto lo vemos en las ilustraciones 1, 2 y 3. Con estos bloques de código solo tuve que darle color para cumplir con lo requerido del ejercicio: La D la coloreamos de rojo, la G de verde y la S de azul. Para renderizarlas, utilicé `shaderList[1]` y para evitar que interfirieran con las figuras del ejercicio 2, utilicé `glm::translate` y `glm::scale` para mover mis iniciales hacia arriba, como se ve en el código de la ilustración 4 y la ejecución de la ilustración 5.

```
251 //indice 8
252
253 GLfloat vertices_iniciales[] = {
254     // D
255     // Bloque 1
256     -0.8f, 0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
257     -0.7f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
258     -0.8f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
259     -0.7f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
260     -0.7f, 0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
261     -0.8f, 0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
262     // Bloque 2
263     -0.7f, 0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
264     -0.4f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
265     -0.7f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
266     -0.4f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
267     -0.4f, 0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
268     -0.7f, 0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
269     // Bloque 3
270     -0.4f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
271     -0.3f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
272     -0.4f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
273     -0.3f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
274     -0.3f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
275     -0.4f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
276     // Bloque 4
277     -0.7f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
278     -0.4f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
279     -0.7f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
280     -0.4f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
281     -0.4f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
282     -0.7f, -0.4f, 0.0f, 1.0f, 0.0f, 0.0f,
```

Ilustración 1: Vértices de la letra D.

```

283
284 // G
285 // Bloque 1
286 -0.1f, 0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
287 0.2f, 0.3f, 0.0f, 0.0f, 1.0f, 0.0f,
288 -0.1f, 0.3f, 0.0f, 0.0f, 1.0f, 0.0f,
289 0.2f, 0.3f, 0.0f, 0.0f, 1.0f, 0.0f,
290 0.2f, 0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
291 -0.1f, 0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
292 // Bloque 2
293 -0.2f, 0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
294 -0.1f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
295 -0.2f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
296 -0.1f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
297 -0.1f, 0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
298 -0.2f, 0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
299 // Bloque 3
300 -0.1f, -0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
301 0.2f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
302 -0.1f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
303 0.2f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
304 0.2f, -0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
305 -0.1f, -0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
306 // Bloque 4
307 0.1f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
308 0.2f, -0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
309 0.1f, -0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
310 0.2f, -0.4f, 0.0f, 0.0f, 1.0f, 0.0f,
311 0.2f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
312 0.1f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
313 // Bloque 5
314 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
315 0.1f, -0.1f, 0.0f, 0.0f, 1.0f, 0.0f,
316 0.0f, -0.1f, 0.0f, 0.0f, 1.0f, 0.0f,
317 0.1f, -0.1f, 0.0f, 0.0f, 1.0f, 0.0f,
318 0.1f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
319 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
320

```

Ilustración 2: Vértices de la letra G.

```

321 // S
322 // Bloque 1
323 0.3f, 0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
324 0.7f, 0.3f, 0.0f, 0.0f, 0.6f, 1.0f,
325 0.3f, 0.3f, 0.0f, 0.0f, 0.6f, 1.0f,
326 0.7f, 0.3f, 0.0f, 0.0f, 0.6f, 1.0f,
327 0.7f, 0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
328 0.3f, 0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
329 // Bloque 2
330 0.3f, 0.3f, 0.0f, 0.0f, 0.6f, 1.0f,
331 0.4f, 0.0f, 0.0f, 0.0f, 0.6f, 1.0f,
332 0.3f, 0.0f, 0.0f, 0.0f, 0.6f, 1.0f,
333 0.4f, 0.0f, 0.0f, 0.0f, 0.6f, 1.0f,
334 0.4f, 0.3f, 0.0f, 0.0f, 0.6f, 1.0f,
335 0.3f, 0.3f, 0.0f, 0.0f, 0.6f, 1.0f,
336 // Bloque 3
337 0.3f, 0.0f, 0.0f, 0.0f, 0.6f, 1.0f,
338 0.7f, -0.1f, 0.0f, 0.0f, 0.6f, 1.0f,
339 0.3f, -0.1f, 0.0f, 0.0f, 0.6f, 1.0f,
340 0.7f, -0.1f, 0.0f, 0.0f, 0.6f, 1.0f,
341 0.7f, 0.0f, 0.0f, 0.0f, 0.6f, 1.0f,
342 0.3f, 0.0f, 0.0f, 0.0f, 0.6f, 1.0f,
343 // Bloque 4
344 0.6f, -0.1f, 0.0f, 0.0f, 0.6f, 1.0f,
345 0.7f, -0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
346 0.6f, -0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
347 0.7f, -0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
348 0.7f, -0.1f, 0.0f, 0.0f, 0.6f, 1.0f,
349 0.6f, -0.1f, 0.0f, 0.0f, 0.6f, 1.0f,
350 // Bloque 5
351 0.3f, -0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
352 0.7f, -0.5f, 0.0f, 0.0f, 0.6f, 1.0f,
353 0.3f, -0.5f, 0.0f, 0.0f, 0.6f, 1.0f,
354 0.7f, -0.5f, 0.0f, 0.0f, 0.6f, 1.0f,
355 0.7f, -0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
356 0.3f, -0.4f, 0.0f, 0.0f, 0.6f, 1.0f,
357 };
358
359 MeshColor* iniciales = new MeshColor();
360 iniciales->CreateMeshColor(vertices_iniciales, 504); // 84 vértices * 6 atributos
361 meshColorList.push_back(iniciales);

```

Ilustración 3: Vértices de la letra S.

```
// Para el ejercicio 1 de las iniciales

shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

model = glm::mat4(1.0f);
// Posicionar en la parte superior para que no interfiera con el resto del escenario
model = glm::translate(model, glm::vec3(0.0f, 0.45f, -3.0f));
model = glm::scale(model, glm::vec3(0.65f, 0.65f, 1.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

// Renderizar la malla de las iniciales (índice 8 en meshColorList)
meshColorList[8]->RenderMeshColor();
```

Ilustración 4: Código en el while para renderizar las iniciales en pantalla.



Ilustración 5: Ejecución del ejercicio 1 en perspectiva ortográfica.

EJERCICIO 2

Descripción: Generar el dibujo de las 3 figuras de clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear shaders diferentes de los colores: verde, azul, café y magenta en lugar de usar el shader con el color clamp. Se debe de entregar los archivos de shader diferentes dentro de un zip adicional al main y documento escrito.

Solución: Para la creación de los shaders con los colores requeridos, fue necesario ir a la carpeta en donde se almacenan los mismo y crear los archivos con extensión “.frag”, como se observa en la ilustración 6. Después en la

ilustración 7 creamos las variables para los shaders recientemente creados y en la ilustración 8 las instanciamos para que los podamos usar en el while.

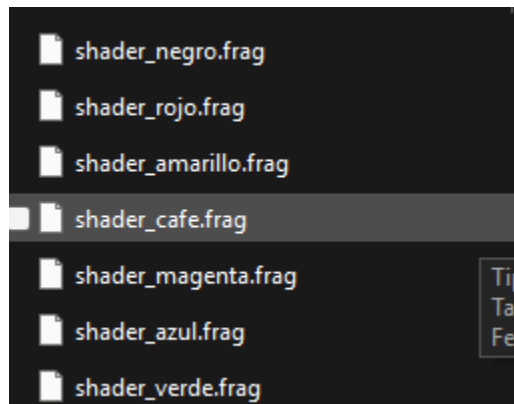


Ilustración 6: Carpeta que almacena los Shaders.

```
28
29 //shaders nuevos se crearían acá
30 static const char* fShaderAzul = "shaders/shader_azul.frag";
31 static const char* fShaderCafe = "shaders/shader_cafe.frag";
32 static const char* fShaderMagenta = "shaders/shader_magenta.frag";
33 static const char* fShaderVerde = "shaders/shader_verde.frag";
34 static const char* fShaderRojo = "shaders/shader_rojo.frag";
35 static const char* fShaderAmarillo = "shaders/shader_amarillo.frag";
36 static const char* fShaderNegro = "shaders/shader_negro.frag";
37
```

Ilustración 7: Creación de las variables para los shaders.

```
void CreateShaders()
{
    Shader *shader1 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shader1->CreateFromFiles(vShader, fShader);
    shaderList.push_back(*shader1); // Índice 0

    Shader *shader2 = new Shader(); //shader para usar color como parte del VAO: letras
    shader2->CreateFromFiles(vShaderColor, fShaderColor);
    shaderList.push_back(*shader2); // Índice 1

    Shader* shaderAzul = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderAzul->CreateFromFiles(vShader, fShaderAzul);
    shaderList.push_back(*shaderAzul); // Índice 2

    Shader* shaderCafe = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderCafe->CreateFromFiles(vShader, fShaderCafe);
    shaderList.push_back(*shaderCafe); // Índice 3

    Shader* shaderMagenta = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderMagenta->CreateFromFiles(vShader, fShaderMagenta);
    shaderList.push_back(*shaderMagenta); // Índice 4

    Shader* shaderVerde = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderVerde->CreateFromFiles(vShader, fShaderVerde);
    shaderList.push_back(*shaderVerde); // Índice 5

    Shader* shaderRojo = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderRojo->CreateFromFiles(vShader, fShaderRojo);
    shaderList.push_back(*shaderRojo); // Índice 6

    Shader* shaderAmarillo = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderAmarillo->CreateFromFiles(vShader, fShaderAmarillo);
    shaderList.push_back(*shaderAmarillo); // Índice 7

    Shader* shaderNegro = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shaderNegro->CreateFromFiles(vShader, fShaderNegro);
    shaderList.push_back(*shaderNegro); // Índice 8
}
```

Ilustración 8: Código para crear los shaders para ser usados en el While.

Después generamos el código para crear las pirámides triangulares y los cubos en las posiciones adecuadas (ilustraciones 9, 10, 11 y 12), siendo, otra vez, el holocrón la figura que más dolores de cabeza me dio. Aunque pregunté para ver cómo podría acomodarlo, hubo algunos errores y tuve que estar probando por ensayo y error, lo bueno es que podíamos usar la proyección ortográfica, ya que en la proyección por perspectiva me demoraba más en acomodar las figuras en una posición que me gustara. Esto último se mira reflejado en las ilustraciones 13 y 14.

```

346
347 //PISO NEGRO
348 Shader shaderNegro = shaderList[8];
349 shaderNegro.useShader();
350 uniformModel = shaderNegro.getModelLocation();
351 uniformProjection = shaderNegro.getProjectLocation();
352
353 model = glm::mat4(1.0f);
354 model = glm::translate(model, glm::vec3(0.0f, -0.8f, -3.0f));
355 model = glm::scale(model, glm::vec3(2.0f, 0.1f, 1.0f));
356 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
357 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
358 meshList[1]-->RenderMesh(); // Cubo
359

```

Ilustración 9: Código para el piso negro

```

// TORRE DE LANZAMIENTO
Shader shaderCafe = shaderList[3];
Shader shaderAmarillo = shaderList[7];
Shader shaderRojo = shaderList[6];
Shader shaderVerde = shaderList[5];
// Pilar Izquierdo (Cubo Café)
shaderCafe.useShader();
uniformModel = shaderCafe.getModelLocation();
uniformProjection = shaderCafe.getProjectLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.8f, -0.35f, -3.0f));
model = glm::scale(model, glm::vec3(0.06f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]-->RenderMesh();
// Pilar Derecho (Cubo Café)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.48f, -0.35f, -3.0f));
model = glm::scale(model, glm::vec3(0.06f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]-->RenderMesh();
// Pirámide triangular Amarillo
shaderAmarillo.useShader();
uniformModel = shaderAmarillo.getModelLocation();
uniformProjection = shaderAmarillo.getProjectLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.65f, -0.15f, -3.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.26f, 0.24f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]-->RenderMesh();
// Pirámide triangular Rojo
shaderRojo.useShader();
uniformModel = shaderRojo.getModelLocation();
uniformProjection = shaderRojo.getProjectLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.65f, -0.39f, -3.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.26f, 0.24f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]-->RenderMesh();
// Pirámide triangular Verde
shaderVerde.useShader();
uniformModel = shaderVerde.getModelLocation();
uniformProjection = shaderVerde.getProjectLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.65f, -0.63f, -3.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.26f, 0.24f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]-->RenderMesh();

```

Ilustración 10: Código para la torre lanzacohetes.

```

PracticasLabCompuGrafica

//Hologrón
ShaderK shaderKagata = shaderList[4];
ShaderK shaderKzul = shaderList[2];
// Pirámide triangular Amarillo
shaderAmarillo.useShader();
uniformModel = shaderAmarillo.getModelLocation();
uniformProjection = shaderAmarillo.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-s, sy + s, -3.02f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.16f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 0.18f, 1.0f));
glUniformMatrix4fv(shaderAmarillo, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderAmarillo, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Pirámide triangular Rojo
shaderRojo.useShader();
uniformModel = shaderRojo.getModelLocation();
uniformProjection = shaderRojo.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(s, sy + s, -3.02f));
model = glm::rotate(model, glm::radians(115.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.16f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 0.18f, 1.0f));
glUniformMatrix4fv(shaderRojo, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderRojo, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Pirámide triangular Magenta
shaderMagenta.useShader();
uniformModel = shaderMagenta.getModelLocation();
uniformProjection = shaderMagenta.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-s, sy - s, -3.02f));
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.16f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 0.18f, 1.0f));
glUniformMatrix4fv(shaderMagenta, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderMagenta, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Pirámide triangular Verde
shaderVerde.useShader();
uniformModel = shaderVerde.getModelLocation();
uniformProjection = shaderVerde.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(s, sy - s, -3.02f));
model = glm::rotate(model, glm::radians(225.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.16f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 0.18f, 1.0f));
glUniformMatrix4fv(shaderVerde, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderVerde, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Cubo Azul
shaderAzul.useShader();
uniformModel = shaderAzul.getModelLocation();
uniformProjection = shaderAzul.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, sy, -3.01f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.35f, 0.18f, 1.0f));
glUniformMatrix4fv(shaderAzul, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderAzul, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1] = RenderMesh(); // Cubo
// Cubo Cafe
shaderCafe.useShader();
uniformModel = shaderCafe.getModelLocation();
uniformProjection = shaderCafe.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, sy, -3.08f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));

```

Ilustración 11: Código para el hologrón.

```

// Pirámides cuadrangular
// Pirámide triangular Verde
shaderVerde.useShader();
uniformModel = shaderVerde.getModelLocation();
uniformProjection = shaderVerde.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.50f, -0.60f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(shaderVerde, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderVerde, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Pirámide triangular Rojo
shaderRojo.useShader();
uniformModel = shaderRojo.getModelLocation();
uniformProjection = shaderRojo.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.80f, -0.60f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(shaderRojo, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderRojo, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Pirámide triangular Magenta
shaderMagenta.useShader();
uniformModel = shaderMagenta.getModelLocation();
uniformProjection = shaderMagenta.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.65f, -0.30f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(shaderMagenta, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderMagenta, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();
// Pirámide triangular Amarillo
shaderAmarillo.useShader();
uniformModel = shaderAmarillo.getModelLocation();
uniformProjection = shaderAmarillo.getProjectionLocation();
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.65f, -0.60f, -3.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 1.0f));
glUniformMatrix4fv(shaderAmarillo, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(shaderAmarillo, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] = RenderMesh();

```

Ilustración 12: Código para la pirámide cuadrangular.

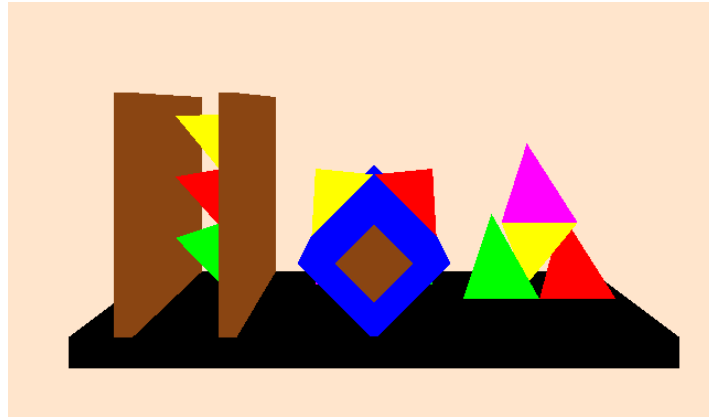


Ilustración 13: Ejecución del ejercicio 2 en proyección de perspectiva.

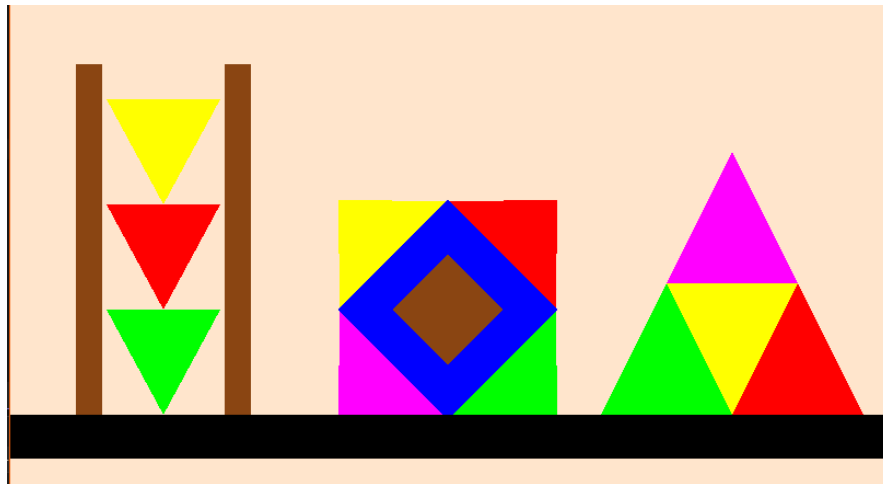


Ilustración 14: Ejecución del ejercicio 2 en proyección ortográfica.

CONCLUSIÓN

Aunque estuve una cantidad importante de tiempo revisando las transformaciones geométricas compuestas (como es la traslación, rotación y escalamiento), aun me faltan comprenderlas para que pueda usarlas de manera más sencilla. Asimismo, el ejercicio 2 fue más llevadero al poder utilizar la proyección ortográfica, ya que me demostrando permitió la alineación precisa de elementos en un plano bidimensional controlado. Aunque me dejó con curiosidad de poder utilizar la proyección por perspectiva.